Data Visualization Final Project Report

Chengkai Kang
ck4193
ck4193@nyu.edu

Abstract

This project presents an interactive web-based visualization dashboard for exploring NYC 311 noise complaint data. The system transforms a large public dataset into coordinated visual views that help users identify spatial, categorical, neighborhood-level, and temporal complaint patterns. The dashboard includes a geo heatmap, complaint type breakdown, neighborhood ranking chart, temporal heatmap, and interactive filters for exploring subsets of the data. By combining preprocessing, map-based visualization, time-based aggregation, and React-based interaction, the project allows users to understand where noise complaints are concentrated, which complaint types are most common, which local areas report the most complaints, and how complaint activity varies by day of week and hour of day.

1. Introduction

Noise complaints are a common quality-of-life issue in New York City. The NYC 311 dataset provides a large collection of public service requests, including many records related to residential noise, commercial noise, construction, street activity, vehicles, and other urban sound sources. While the raw dataset is useful, it is difficult to interpret directly because it contains many records across different locations, timestamps, and complaint categories.

The objective of this project is to build an interactive visualization dashboard that makes NYC 311 noise complaint patterns easier to explore. Instead of presenting the data as a static table, the dashboard provides multiple coordinated views. Users can examine complaint density on a geographic heatmap, compare complaint categories using a chart, and identify neighborhoods with the highest complaint counts.

The main design goal is exploratory analysis. The dashboard is not intended to prove a single fixed conclusion. Instead, it allows users to ask questions such as: where are noise complaints most concentrated, which complaint types are most common, and how do these patterns change when filtering by borough or category?

2. Related Work

Urban visual analytics. Interactive visualization has been widely used to explore large-scale urban data. Ferreira et al. [5] demonstrate how visual queries over millions of NYC taxi trips can reveal spatio-temporal mobility patterns that are difficult to observe from aggregate statistics alone. Miranda et al. [7] introduce Urban Pulse, a framework for identifying rhythmic activity patterns in cities across multiple temporal resolutions. Chen et al. [2] present VAUD, a visual analytics system for exploring heterogeneous spatio-temporal urban data across multiple coordinated views. Doraiswamy et al. [3] further summarize challenges in spatio-temporal urban visual analytics, including scalable data management, interactive filtering, and multi-view exploration. These systems motivate the coordinated spatial, categorical, and temporal views used in this project.

Urban noise and soundscape analysis. Urban noise has also been studied through sensor networks, participatory sensing, and machine listening. Maisonneuve et al. [6] propose NoiseTube, a participatory sensing system that turns GPS-equipped mobile phones into tools for measuring and mapping noise exposure. D'Hondt et al. [4] evaluate participatory noise mapping as an alternative to traditional noise assessment methods. In New York City, the SONYC project presents a cyber-physical sensing system for monitoring, analyzing, and mitigating urban noise using acoustic sensors and machine learning [1]. Miranda et al. [8] introduce Time Lattice for interactive analysis of large temporal sensor streams and apply it to urban noise data, while Rulff et al. [10] combine machine listening and visual analytics to explore urban soundscapes. Unlike these sensor-based systems, this project focuses on citizen-generated 311 complaints, which reflect perceived noise disturbances reported by residents.

311 service requests and civic data. 311 service request data has been used to study urban conditions, service demand, and citizen-government interaction. Zheng et al. [15] describe urban computing as the integration of urban sensing, data management, analytics, and visualization. Wang

et al. [12] show that the structure of 311 service requests can act as a signature of urban neighborhoods and can be used to study socioeconomic patterns. Zhang et al. [14] present a visual analytics system for public utility service problems, showing how complaint-style civic data can reveal spatial patterns in service demand. These works inform the interpretation of 311 complaints in this project: complaint counts are useful indicators of reported disturbance, but they should not be treated as direct measurements of objective noise levels.

Coordinated interaction and visual exploration. The dashboard also follows established principles from information visualization. Shneiderman's visual information-seeking mantra emphasizes "overview first, zoom and filter, then details-on-demand," which aligns with the dashboard's structure of summary statistics, filters, and coordinated views [11]. Weaver [13] describes cross-filtered views for multidimensional visual analysis, where selections in one view update related views. This project applies the same idea to NYC 311 noise data: selecting a borough, complaint type, ZIP-code area, temporal cell, or map region updates the other visualizations through shared filtering logic.

These works inform the interpretation of 311 complaints in this project: complaint counts are useful indicators of reported disturbance, but they should not be treated as direct measurements of objective noise levels.

3. Data and Preprocessing

The raw complaint records come from NYC Open Data's 311 Service Requests dataset [9].

The preprocessing pipeline was implemented in a Jupyter notebook and follows an API-first workflow. Instead of manually downloading a static CSV file, the notebook retrieves 2025 NYC 311 service request records directly from the NYC Open Data Socrata API. The dataset ID used for the 311 service request data is `erm2-nwe9`. The pipeline downloads records month by month, saves a complete raw archive, builds a smaller curated analysis table, filters to explicit noise complaint categories, and finally exports frontend-ready JSON files for the visualization dashboard.

The decision to use only 2025 records was made for performance and project scope. The full NYC 311 dataset is very large, so limiting the project to one year made the dashboard faster to process and easier to interact with while still preserving enough data to reveal meaningful spatial and categorical patterns.

3.1. API Download and Caching

The notebook downloads 2025 records from the NYC Open Data API one month at a time. Each request uses a date window based on the `created_date` field, along with pagination through `$limit` and `$offset`. This month-by-month approach avoids trying to download the entire year in one request and makes the pipeline easier to resume or debug.

The raw downloaded data is saved as a Parquet file:

- `nyc_311_2025_raw.parquet`: full 2025 raw archive with all available columns.

A cache flag is used so that the notebook can reload the saved Parquet file instead of repeatedly downloading the same API data. This makes later preprocessing and experimentation much faster.

Listing 1. Monthly API download and raw caching setup.

```
DATASET_ID = "erm2-nwe9"
BASE_URL = f"https://data.cityofnewyork.us/
    resource/{DATASET_ID}.csv"

YEAR = 2025
LIMIT = 50_000
USE_CACHE = True

RAW_2025_PATH = RAW_DIR / "nyc_311_2025_raw.
    parquet"
ANALYSIS_2025_PATH = RAW_DIR / "
    nyc_311_2025_analysis.parquet"
NOISE_2025_PATH = RAW_DIR / "nyc_311_noise_2025.
    parquet"
OUTPUT_PATH = OUTPUT_DIR / "complaints.json"
```

3.2. Schema Normalization

One important part of the preprocessing pipeline is schema-safe reading and writing. The notebook first reads downloaded CSV chunks as strings. After loading, known datetime columns such as `created_date` and `closed_date` are explicitly converted to datetime values, while numeric geospatial columns such as `latitude` and `longitude` are converted to numeric values.

This step prevents mixed-type errors when saving to Parquet. For example, fields such as ZIP codes may contain values that look numeric in some rows but should still be treated as strings. The notebook keeps categorical fields, IDs, ZIP codes, and text fields as string values to avoid inconsistent column types.

Listing 2. Schema normalization for Parquet export.

```
def normalize_frame_for_parquet(df: pd.DataFrame)
     -> pd.DataFrame:
    df = df.copy()

    datetime_cols = [
        col for col in [
            "created_date",
            "closed_date",
            "due_date",
            "resolution_action_updated_date",
        ] if col in df.columns
```

```
    ]
    for col in datetime_cols:
        df[col] = pd.to_datetime(df[col], errors=
            "coerce", utc=True)

    numeric_cols = [
        col for col in [
            "latitude",
            "longitude",
            "x_coordinate_state_plane",
            "y_coordinate_state_plane",
        ] if col in df.columns
    ]
    for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors="
            coerce")

    for col in df.columns:
        if pd.api.types.is_object_dtype(df[col]):
            df[col] = df[col].astype("string")

    return df
```

3.3. Curated Analysis Layer

After saving the full raw archive, the notebook creates a smaller curated analysis table containing only the fields needed for the visualization pipeline. This table reduces file size and keeps the later processing steps focused on relevant attributes.

The selected analysis columns include identifiers, timestamps, complaint type, descriptor, borough, ZIP code, community board, latitude, longitude, and status. This curated table is saved separately as:

- `nyc_311_2025_analysis.parquet`: smaller analysis layer used for filtering and visualization exports.

Listing 3. Curated analysis columns.

```
ANALYSIS_COLUMNS = [
    "unique_key",
    "created_date",
    "closed_date",
    "agency",
    "complaint_type",
    "descriptor",
    "borough",
    "incident_zip",
    "city",
    "community_board",
    "open_data_channel_type",
    "latitude",
    "longitude",
    "status",
]
```

3.4. Filtering Noise Complaints

The dashboard focuses specifically on noise-related 311 complaints. Instead of using a broad keyword search for any complaint type containing the word "Noise," the notebook uses an explicit list of accepted noise complaint types. This avoids accidentally including vague or inconsistent complaint categories.

The retained complaint types are:

- `Noise - Commercial`
- `Noise - Helicopter`
- `Noise - House of Worship`
- `Noise - Park`
- `Noise - Residential`
- `Noise - Street/Sidewalk`
- `Noise - Vehicle`

After filtering to these complaint types, the notebook removes records missing required fields such as `created_date`, `latitude`, `longitude`, and `borough`. This ensures that all exported records can be used reliably in both the map and chart visualizations.

Listing 4. Explicit filtering to selected noise complaint types.

```
NOISE_TYPES = [
    "Noise - Commercial",
    "Noise - Helicopter",
    "Noise - House of Worship",
    "Noise - Park",
    "Noise - Residential",
    "Noise - Street/Sidewalk",
    "Noise - Vehicle",
]

complaint_type_normalized = (
    df["complaint_type"]
    .astype("string")
    .str.strip()
)

df = df[complaint_type_normalized.isin(
    NOISE_TYPES)].copy()

df = df.dropna(
    subset=["created_date", "latitude", "
        longitude", "borough"]
)
```

3.5. Feature Engineering

The notebook adds several derived fields used by the frontend visualizations. From the `created_date` timestamp, the pipeline extracts year, month, hour, day of week, and date. These fields support filtering and time-based aggregation.

The notebook also creates a temporary `neighborhood` field. In the current version, this field

uses `incident_zip` as a placeholder for neighborhood-level grouping. This allows the dashboard to support a neighborhood ranking visualization, although a future version could replace this with a more accurate spatial join to NYC Neighborhood Tabulation Areas or community districts.

Listing 5. Time features and placeholder neighborhood field.

```
df["year"] = df["created_date"].dt.year
df["month"] = df["created_date"].dt.month
df["day_of_week"] = df["created_date"].dt.
    day_name()
df["hour"] = df["created_date"].dt.hour
df["date"] = df["created_date"].dt.date.astype(
    str)

def infer_neighborhood(row: pd.Series) -> str:
    value = row.get("incident_zip")
    if pd.isna(value) or value in [None, ""]:
        return "Unknown"
    return str(value)

df["neighborhood"] = df.apply(infer_neighborhood,
     axis=1)
```

3.6. Frontend Export Format and Validation

The final preprocessing step standardizes the cleaned data into frontend-friendly field names that match the React components. For example, `created_date` is exported as `createdAt`, `complaint_type` as `complaintType`, and `day_of_week` as `dayOfWeek`. The main point-level export contains one object per complaint record, including the complaint ID, timestamp, complaint type, borough, ZIP-code-based neighborhood placeholder, latitude, longitude, status, and derived time fields.

The notebook exports several files for different dashboard components, as shown in Table 1. This separation keeps the frontend simpler because each visualization can load data that is already close to the format it needs. The point dataset supports the geo heatmap and dashboard filters, the complaint type count file supports the category breakdown, the neighborhood statistics file supports the local ranking view, and the temporal matrix supports the day-hour heatmap.

After export, the notebook performs validation checks on the generated data. These checks include reporting the number of unique complaint types, borough-level counts, and missing values by output column. This step confirms that key fields such as coordinates, complaint categories, and frontend field names are valid before the data is used in the dashboard.

4. Dashboard Design

The website is structured as an interactive dashboard that combines filtering controls, summary statistics, and coordinated visualizations. The filter panel allows users to narrow the dataset by borough, complaint type, year range, selected temporal window, and spatial map bounds. When a filter changes, the dashboard recomputes the active complaint records and passes the same filtered dataset to the relevant visualization components.

Table 1. Files produced by the preprocessing notebook.

File	Purpose
`nyc_311_2025_raw.parquet`	Full raw 2025 archive
`nyc_311_2025_analysis.parquet`	Curated analysis table
`nyc_311_noise_2025.parquet`	Filtered noise subset
`complaints_points.json`	Point records for map and filters
`temporal_matrix.json`	Day-hour matrix
`complaint_type_counts.json`	Complaint type summary
`neighborhood_stats.json`	ZIP-code area summary

The dashboard includes four main visualization components:

- **Geo heatmap:** shows geographic density of complaints across NYC.
- **Complaint type breakdown:** compares the frequency of different noise complaint categories.
- **Neighborhood ranking:** ranks ZIP-code-based neighborhood areas by complaint count.
- **Temporal heatmap:** shows complaint volume by day of week and hour of day.

The temporal heatmap adds a time-based perspective to the dashboard. While the geo heatmap shows where complaints are concentrated, the temporal heatmap shows when complaints are most frequent. Each temporal cell represents a day-hour combination, and selecting a cell applies a temporal filter to the rest of the dashboard. This allows users to connect time-based patterns with geographic hotspots, complaint categories, and neighborhood rankings.

The main dashboard coordinates all views through shared filter state. The `filtered` dataset contains records that match the active borough, complaint type, neighborhood, spatial, year, and temporal selections. This dataset is passed to the geo heatmap, complaint type chart, and neighborhood ranking. The temporal heatmap receives `filteredWithoutTemporal`, which removes the currently selected temporal cell before rebuilding the day-hour matrix. This prevents the temporal heatmap from collapsing to a single selected cell while still allowing the selected cell to filter the other dashboard views.

Listing 6. Dashboard-level filtering and coordinated visualization inputs.

```
const filtered = useMemo(() => {
  if (!data) return [];
  return filterComplaints(data.points, filters);
}, [data, filters]);

const filteredWithoutTemporal = useMemo(() => {
  if (!data) return [];

  return filterComplaints(data.points, {
    ...filters,
    selectedTemporalCell: null,
  });
}, [data, filters]);
```

4.1. Dashboard Overview

Figure 1 shows the overall dashboard interface for the Brooklyn subset of the 2025 NYC 311 noise complaint data. The dashboard combines summary statistics, interactive filters, and coordinated visualizations in a single view. Users can filter by borough, complaint type, year range, map region, and selected temporal cells. These selections update the geo heatmap, complaint type breakdown, neighborhood ranking, and temporal heatmap together.

5. Results and Demonstration

This section presents the main visual components of the dashboard and explains how each one supports data exploration.

5.1. Geo Heatmap

The geo heatmap is the central map-based visualization. Its purpose is to show where noise complaints are geographically concentrated across New York City. Each complaint record contains latitude and longitude coordinates. Instead of drawing every complaint as an individual marker, the dashboard uses a heatmap layer to blend nearby points into a density surface.

This makes the visualization more readable for large datasets. Individual markers would create clutter, especially in dense areas of Manhattan, Brooklyn, and Queens. The heatmap makes high-density complaint regions more visible through stronger color intensity. As shown in Figure 2, the heatmap helps reveal geographic hotspots that are less obvious in a table of individual records.

The geographic heatmap is implemented in `NoiseHeatmap.tsx` using DeckGL's `HeatmapLayer` on top of a Mapbox basemap. Each complaint record is passed directly to the heatmap layer, where `getPosition` converts the record into a longitude-latitude coordinate pair. Each complaint receives a uniform weight of one, so the resulting heatmap represents complaint density rather than severity. The layer parameters, including radius, intensity, and threshold, control how nearby points are blended into a continuous spatial density surface.

Listing 7. Geo heatmap layer implementation using DeckGL.

```
const layers = useMemo(() => {
  return [
    new HeatmapLayer<ComplaintRecord>({
      id: 'noise-heatmap',
      data: records,
      getPosition: (d) => [d.longitude, d.
          latitude],
      getWeight: () => 1,
      radiusPixels: 45,
      intensity: 1,
      threshold: 0.03,
    }),
  ];
}, [records]);
```

The heatmap also responds to the dashboard filters. For example, selecting a specific borough or complaint type updates the map to display only the matching complaints.

5.2. Complaint Type Chart

The complaint type breakdown summarizes the number of complaints in each selected noise category. This view helps users compare the relative frequency of residential, street, commercial, vehicle, and other noise complaint types. In the Brooklyn 2025 view shown in Figure 3, residential noise clearly dominates the dataset, followed by street or sidewalk noise and commercial noise. The chart is interactive, so selecting a category applies that complaint type as a dashboard-wide filter.

The complaint type chart uses a hybrid data strategy. When the dashboard is showing the full dataset, it uses precomputed complaint type counts exported by the preprocessing notebook. When filters are active, it recomputes the breakdown from the currently filtered records using `buildComplaintTypeBreakdown`. The resulting data is sorted by count and rendered as a horizontal bar list, where bar width is scaled relative to the largest category.

Listing 8. Complaint type breakdown data selection.

```
const derivedData = useMemo(
  () => buildComplaintTypeBreakdown(records),
  [records]
);

const data = useMemo(() => {
  const usePrecomputed = records.length ===
      totalPointCount;

  if (usePrecomputed) {
    return precomputedCounts
      .map((item) => ({
        label: item.complaintType,
        value: item.count,
      }))
      .sort((a, b) => b.value - a.value);
  }
```

Figure 1. Dashboard overview showing the Brooklyn 2025 noise complaint explorer interface.

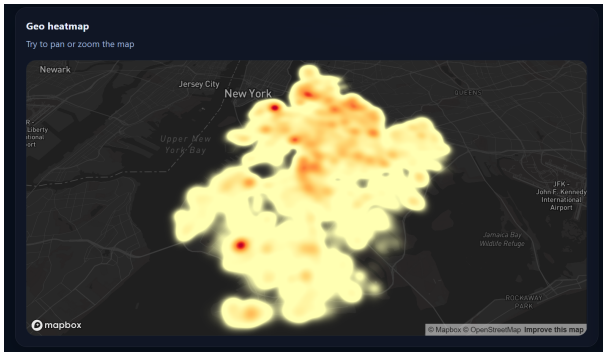

Figure 2. Geo heatmap showing the spatial distribution of noise complaints in Brooklyn in 2025.

```
  return derivedData;
}, [records.length, totalPointCount,
    precomputedCounts, derivedData]);
```

5.3. Neighborhood Ranking

The neighborhood ranking view provides a local comparison of complaint volume across ZIP-code-based neighborhood placeholders. While the geo heatmap is useful for identifying spatial clusters visually, the ranking chart makes it easier to compare locations directly. Figure 4 shows the highest complaint-count areas in Brooklyn in 2025, allowing users to quickly identify which local regions generate the most noise reports.

The neighborhood ranking component follows the same precomputed-versus-derived pattern. For the unfiltered dataset, it uses precomputed neighborhood statistics from the preprocessing output. Once filters are applied, the ranking is rebuilt from the active record set. The component

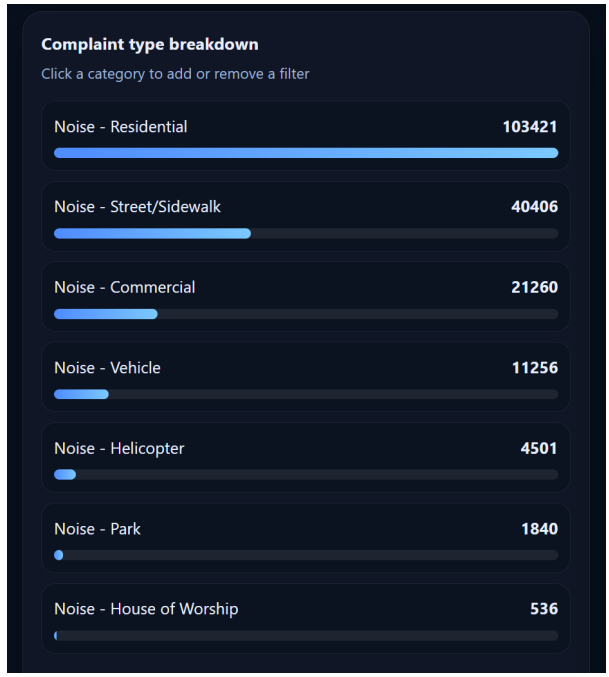

Figure 3. Complaint type breakdown showing noise complaint counts for Brooklyn in 2025.

sorts locations by complaint count and displays the top results as scaled horizontal bars. In the current preprocessing pipeline, the neighborhood field is based on ZIP-code-style area labels, so the chart should be interpreted as a ZIP-code-based local ranking rather than an official neighborhood boundary analysis.

Listing 9. Neighborhood ranking data selection.

```
const derivedData = useMemo(
  () => buildNeighborhoodRanking(records),
```

Figure 4. Neighborhood ranking showing the highest complaint-count ZIP-code areas in Brooklyn in 2025.

```
  [records]
);

const data = useMemo(() => {
  const usePrecomputed = records.length ===
      totalPointCount;

  if (usePrecomputed) {
    return precomputedStats
      .map((item) => ({
        label: item.neighborhood,
        value: item.count,
      }))
      .sort((a, b) => b.value - a.value)
      .slice(0, 15);
  }

  return derivedData;
}, [records.length, totalPointCount,
    precomputedStats, derivedData]);
```

5.4. Temporal Heatmap

The temporal heatmap summarizes when noise complaints occur most frequently. The visualization aggregates complaint records by day of week and hour of day, producing a two-dimensional matrix in which each cell represents the complaint volume for a specific day-hour combination. Color intensity encodes the relative number of complaints, with cooler colors representing lower counts and warmer colors representing higher counts.

This view is useful because it reveals recurring temporal patterns that are difficult to identify from raw records alone. In the Brooklyn 2025 filtered view shown in Figure 5, higher complaint activity appears during evening and late-night periods, especially on weekends. The heatmap is also interactive: selecting a cell applies a day-hour filter to the rest of the dashboard, allowing users to connect temporal patterns with the map, complaint type chart, and neighborhood ranking.

The component can either use a precomputed matrix exported by the preprocessing notebook or rebuild the matrix from the active filtered records. The precomputed path is used when the full dataset is displayed, while filtered views are derived directly from the current record set.

Listing 10. Building a day-hour temporal matrix from precomputed counts.

```
function buildMatrixFromPrecomputed(
    precomputedMatrix) {
  const matrix = Array.from({ length: 7 }, (_,
      day) =>
    Array.from({ length: 24 }, (_, hour) => ({
      day,
      hour,
      value: 0,
    })),
  );

  for (const row of precomputedMatrix) {
    const dayIndex = dayToIndex(row.dayOfWeek);
    if (dayIndex === null) continue;
    if (row.hour < 0 || row.hour > 23) continue;

    matrix[dayIndex][row.hour] = {
      day: dayIndex,
      hour: row.hour,
      value: row.count,
    };
  }

  return matrix;
```

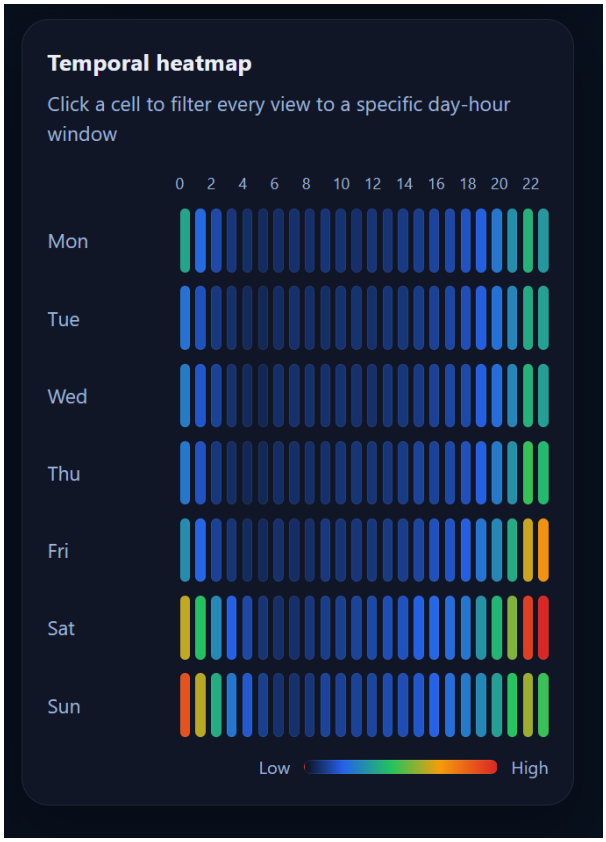

Figure 5. Temporal heatmap showing noise complaint volume by day of week and hour of day for Brooklyn in 2025.

```
}
```

6. Implementation

The system is divided into two main stages: an offline preprocessing pipeline and an interactive React frontend. The preprocessing notebook downloads and cleans the 2025 NYC 311 data, filters it to selected noise complaint types, derives time and location fields, and exports frontend-ready JSON files into the public data directory. The React dashboard then loads these processed files instead of querying the raw NYC 311 API directly, which improves performance and keeps the visualization components simpler.

The frontend uses four main data files: point-level complaint records, a temporal matrix, complaint type counts, and neighborhood statistics. The point-level records contain the fields shared across the dashboard, including complaint ID, timestamp, complaint type, borough, ZIP-code-based neighborhood label, latitude, longitude, status, and derived time fields. The aggregated files support the temporal heatmap, complaint type chart, and neighborhood ranking.

Listing 11. Loading preprocessed JSON files into the dashboard.

```
export async function loadAppData(): Promise<
    AppData> {
  const [points, temporal, complaintTypes,
      neighborhoods] = await Promise.all([
    loadJson<ComplaintRecord[]>('data/
        complaints_points.json'),
    loadJson<TemporalMatrixRecord[]>('data/
        temporal_matrix.json'),
    loadJson<ComplaintTypeCountRecord[]>('data/
        complaint_type_counts.json'),
    loadJson<NeighborhoodStatRecord[]>('data/
        neighborhood_stats.json'),
  ]);

  return {
    points,
    temporal,
    complaintTypes,
    neighborhoods,
  };
}
```

Dashboard interaction is controlled through a shared filter state. This state tracks the selected borough, complaint types, neighborhoods, temporal day-hour cell, year range, and optional spatial bounds from the map. Because all visualizations use this same filtering logic, selecting an item in one view updates the rest of the dashboard consistently. For example, clicking a complaint type updates the map, ranking chart, and temporal heatmap to reflect only the selected category.

The core filtering function applies the active dashboard selections to the point-level complaint records. The snippet below shows the main filtering logic for borough, complaint type, neighborhood, year range, and temporal cell selections. The full implementation also supports spatial bounds from the map view. The resulting filtered record set becomes the shared input for the geo heatmap, complaint type breakdown, and neighborhood ranking.

Listing 12. Filtering complaint records based on dashboard state.

```
export function filterComplaints(records, filters
    ) {
  return records.filter((record) => {
    if (filters.selectedBorough !== 'All' &&
        record.borough !== filters.
            selectedBorough) {
      return false;
    }

    if (filters.selectedComplaintTypes.length > 0
         &&
        !filters.selectedComplaintTypes.includes(
            record.complaintType)) {
      return false;
    }

    if (filters.selectedNeighborhoods.length > 0
        &&
        !filters.selectedNeighborhoods.includes(
            record.neighborhood)) {
```

```
      return false;
    }

    if (filters.yearRange) {
      const year = getDateParts(record.createdAt)
          .year;
      if (year < filters.yearRange[0] || year >
          filters.yearRange[1]) {
        return false;
      }
    }

    if (!matchesTemporalCell(record, filters.
        selectedTemporalCell)) {
      return false;
    }

    return true;
  });
}
```

7. Discussion

The completed dashboard demonstrates how interactive visualization can make a large public dataset easier to understand. The geo heatmap reveals spatial hotspots, the complaint type chart summarizes category-level patterns, the neighborhood ranking shows local complaint concentration, and the temporal heatmap highlights recurring day-hour patterns.

One important insight from the project is that different visualizations reveal different aspects of the same data. The map is useful for identifying geographic clusters, but it is less precise for exact comparison. The ranking chart provides clearer comparison between local areas, while the complaint type chart helps explain what kinds of noise issues are most common. The temporal heatmap adds another perspective by showing when complaint activity is concentrated.

A limitation of the current system is that raw complaint counts do not account for neighborhood population, area size, land use, or reporting behavior. Some areas may have more complaints because they have more residents, higher density, more nightlife or commercial activity, or a greater tendency to report issues through 311. Future work could add normalized measures such as complaints per capita or complaints per square mile.

Another possible improvement is to replace the ZIP-code-based neighborhood placeholder with official NYC neighborhood or community district boundaries. This would make the neighborhood ranking more geographically precise. The dashboard could also support additional temporal comparisons, such as month-to-month trends or weekday-versus-weekend summaries.

8. Conclusion

This project built an interactive dashboard for exploring NYC 311 noise complaints. The system combines data preprocessing, React-based frontend development, map-based visualization, coordinated filtering, and temporal aggregation. The final dashboard helps users identify spatial complaint hotspots, compare complaint categories, examine ZIP-code-based local rankings, and analyze complaint volume by day of week and hour of day.

Overall, the project shows how visualization can transform a large public dataset into a more accessible exploratory tool. By presenting the data through coordinated spatial, categorical, local, and temporal views, the dashboard makes it easier to understand patterns in urban noise complaints across New York City.

References

[1] Juan Pablo Bello, Claudio Silva, Oded Nov, R. Luke DuBois, Anish Arora, Justin Salamon, Charles Mydlarz, and Harish Doraiswamy. Sonyc: A system for the monitoring, analysis and mitigation of urban noise pollution, 2018. 1

[2] Wei Chen, Zhaosong Huang, Feiran Wu, Minfeng Zhu, Huihua Guan, and Ross Maciejewski. Vaud: A visual analysis approach for exploring spatio-temporal urban data. *IEEE Transactions on Visualization and Computer Graphics*, 24(9):2636–2648, 2018. 1

[3] Harish Doraiswamy, Juliana Freire, Marcos Lage, Fabio Miranda, and Claudio Silva. Spatio-temporal urban data analysis: A visual analytics perspective. *IEEE Computer Graphics and Applications*, 38(5):26–35, 2018. 1

[4] Ellie D'Hondt, Matthias Stevens, and An Jacobs. Participatory noise mapping works! an evaluation of participatory sensing as an alternative to standard techniques for environmental monitoring. *Pervasive and Mobile Computing*, 9(5):681–694, 2013. Special issue on Pervasive Urban Applications. 1

[5] Nivan Ferreira, Jorge Poco, Huy T. Vo, Juliana Freire, and Cláudio T. Silva. Visual exploration of big spatio-temporal urban data: A study of new york city taxi trips. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2149–2158, 2013. 1

[6] Nicolas Maisonneuve, Matthias Stevens, Maria Niessen, and Luc Steels. Noisetube: Measuring and mapping noise pollution with mobile phones. pages 215–228, 01 2009. 1

[7] Fabio Miranda, Harish Doraiswamy, Marcos Lage, Kai Zhao, Bruno Goncalves, Luc Wilson, Mondrian Hsieh, and Claudio T. Silva. Urban pulse: Capturing the rhythm of cities. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):791–800, Jan. 2017. 1

[8] Fabio Miranda, Marcos Lage, Harish Doraiswamy, Charlie Mydlarz, Justin Salamon, Yitzchak Lockerman, Juliana Freire, and Claudio T. Silva. Time lattice: A data structure for the interactive visual analysis of large time series. *Computer Graphics Forum*, 37(3):23–35, 2018. 1

[9] NYC Open Data. NYC open data: 311 service requests. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9, 2024. 2

[10] Joao Rulff, Fabio Miranda, Maryam Hosseini, Marcos Lage, Mark Cartwright, Graham Dove, Juan Bello, and Claudio T. Silva. Urban rhapsody: Large-scale exploration of urban soundscapes. *Computer Graphics Forum*, 41(3):209–221, June 2022. 1

[11] B. Shneiderman. The eyes have it: a task by data type taxonomy for information visualizations. In *Proceedings 1996 IEEE Symposium on Visual Languages*, pages 336–343, 1996. 2

[12] Lingjing Wang, Cheng Qian, Philipp Kats, Constantine Kontokosta, and Stanislav Sobolevsky. Structure of 311 service requests as a signature of urban location. *PLOS ONE*, 12(10):1–21, 10 2017. 2

[13] Chris Weaver. Cross-filtered views for multidimensional visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 16(2):192–204, 2010. 2

[14] Jiawan Zhang, E Yanli, Jing Ma, Yahui Zhao, Binghan Xu, Liting Sun, Jinyan Chen, and Xiaoru Yuan. Visual analysis of public utility service problems in a metropolis. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):1843–1852, 2014. 2

[15] Yu Zheng, Licia Capra, Ouri Wolfson, and Hai Yang. Urban computing: Concepts, methodologies, and applications. *ACM Trans. Intell. Syst. Technol.*, 5(3), Sept. 2014. 1

A. Code Repository

The project source code, preprocessing scripts, and setup instructions are available in the GitHub repository:

https://github.com/Asai-Yume/CSGY-6313-Data-Visualization-Final

Large generated data files are not included in the repository because of GitHub file size limits. The data can be downloaded separately from the provided Google Drive link or regenerated using the preprocessing notebook included in the repository.